

第五章 数组与广义表

5.1 数组的定义

5.2 数组元素的顺序表示和实现

5.3 矩阵的压缩存储

5.4 广义表的定义

5.5 广义表的存储结构

数组是一种数据结构

- 数组是已实现的固有数据类型；
- 数组是一种结构类型，是一种数据结构；
- 数组可以看成是一种特殊的线性表，即线性表中数据元素本身也是一个线性表。

数组的类型定义

ADT Array {

数据对象: $D = \{a_{j_1, j_2, \dots, j_i, \dots, j_n} \mid j_i = 0, \dots, b_i - 1, i = 1, 2, \dots, n,$
称 $n(>0)$ 为数组的**维数**, b_i 为数组第 i 维的**长度**, j_i 为数组元素的第 i 维**下标**,
 $a_{j_1, \dots, j_n} \in \text{ElemSet} \}$

数据关系: $R = \{R_1, R_2, \dots, R_n\}$

$R_i = \{ \langle a_{j_1, \dots, j_i, \dots, j_n}, a_{j_1, \dots, j_{i+1}, \dots, j_n} \rangle \mid$
 $0 \leq j_k \leq b_k - 1, 1 \leq k \leq n \text{ 且 } k \neq i,$
 $0 \leq j_i \leq b_i - 2,$

$a_{j_1, \dots, j_i, \dots, j_n}, a_{j_1, \dots, j_{i+1}, \dots, j_n} \in D,$
 $i = 2, \dots, n \}$

- 数组中有 $b_1 \times b_2 \times \dots \times b_n$ 个元素，每个元素都处在 n 个关系中；
- 如一个二维数组

$$D = \{a_{ij} \mid 0 \leq i \leq m-1, 0 \leq j \leq n-1, a_{ij} \in \text{ElemType}\}$$

$$R = \{\text{ROW}, \text{COL}\}$$

其中：

$$\text{ROW} = \{\langle a_{i-1,j}, a_{ij} \rangle \mid i=1, \dots, m-1, 0 \leq j \leq n-1, a_{i-1,j}, a_{ij} \in \text{ElemType}\}$$

$$\text{COL} = \{\langle a_{i,j-1}, a_{ij} \rangle \mid j=1, \dots, n-1, 0 \leq i \leq m-1, a_{i,j-1}, a_{ij} \in \text{ElemType}\}$$

ROW，称作“**行关系**”，COL称作“**列关系**”。

二维数组中共有 $m \times n$ 个元素，每个元素 a_{ij} 同时处于“行”和“列”的两个关系中，它既在“行关系ROW”中是 $a_{i-1,j}$ ($i > 0$) 的后继，又在“列关系COL”中是 $a_{i,j-1}$ ($j > 0$) 的后继。

- 与线性表类似，数组中的每个元素都对应于一组下标，也可以将数组看成是由“一组下标”和“元素值”构成的二元组的集合。

基本操作:

InitArray(&A, n, bound₁, ..., bound_n)

操作结果: 若维数 n 和各维长度合法, 则构造相应的数组 A 。

DestroyArray(&A)

初始条件: 数组 A 已经存在。

操作结果: 销毁数组 A 。

Value(A, &e, index₁, ..., index_n)

初始条件: A 是 n 维数组, e 为元素变量, 随后是 n 个下标值。

操作结果: 若各下标不超界, 则 e 赋值为所指定的 A 的元素值, 并返回 OK。

Assign(&A, e, index₁, ..., index_n)

初始条件: A 是 n 维数组, e 为元素变量, 随后是 n 个下标值。

操作结果: 若下标不超界, 则将 e 的值赋给 A 中指定下标的元素。

} ADT Array

数组被定义, 其维数 n 和每一维的上、下界均不能再变, 数组中元素之间的关系也不再改变。因此数组的基本操作除初始化和结构销毁之外, 只有通过给定的“一组下标”索引取得元素或修改元素的值。

——因此适合使用顺序存储结构

返回

➤ 由于数组类型不作插入和删除的操作，因此只需要通过“顺序映象”得到它的存储结构，借助数据元素在存储器中的相对位置来表示数据元素之间的逻辑关系。

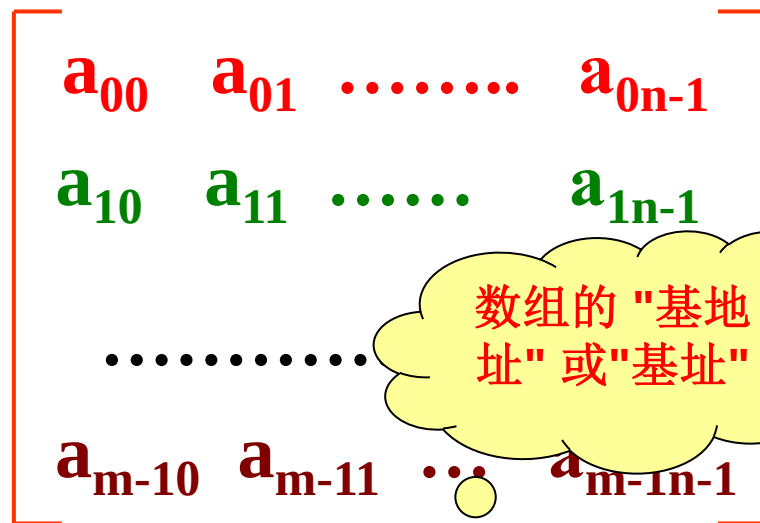
➤ 有两种映象方法：

“以行(序)为主(序)”：对二维数组进行“按行切分”，即将数组中的数据元素“按行依次排放”在存储器中；

“以列(序)为主(序)”：对二维数组进行“按列切分”，即将数组中的数据元素“按列依次排放”在存储器中。

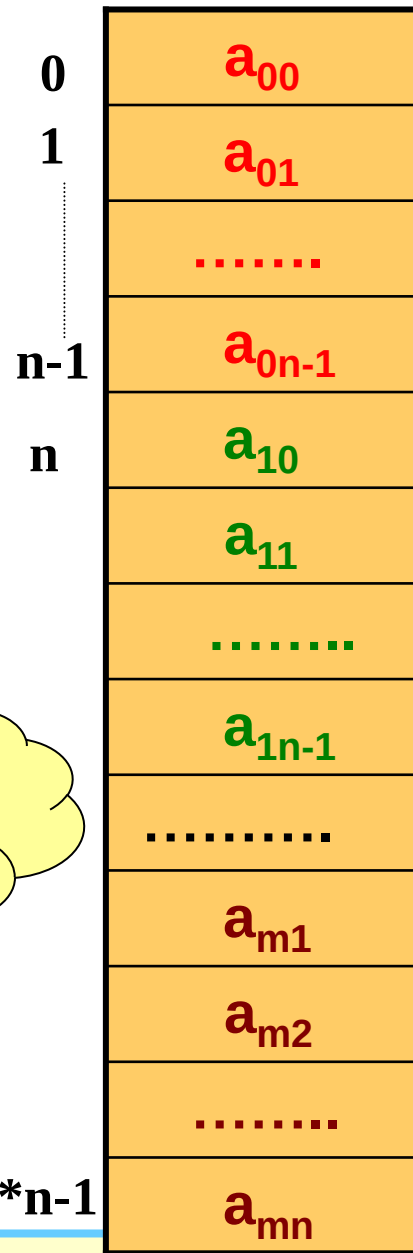
按行序为主序存放

d为单位空间大小



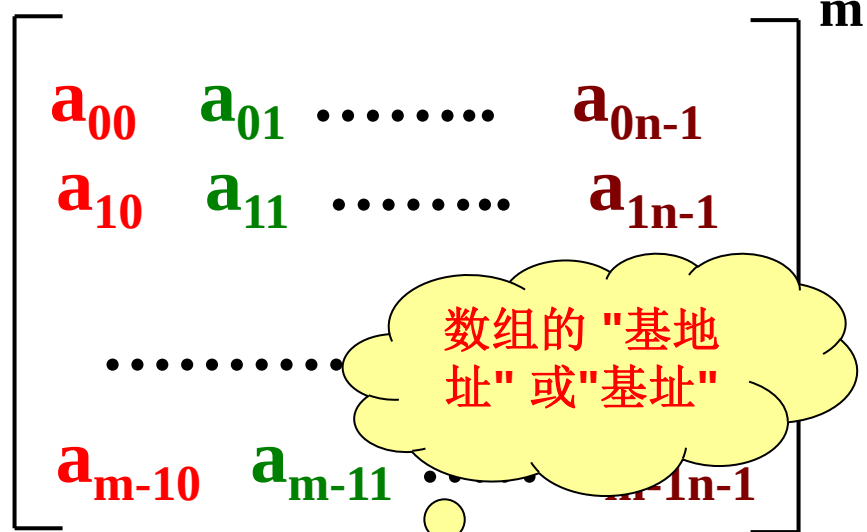
$$\text{Loc}(a_{ij}) = \text{Loc}(a_{00}) + [i * n + j] * d$$

m*n-1



现有的高级语言中的大多数都是按"以行为主"实现数组类型的。

按列序为主序存放



$$\text{Loc}(a_{ij}) = \text{Loc}(a_{00}) + (j * m + i) * d$$

$m * n - 1$

0

a_{00}

1

a_{10}

.....

m-1

a_{m-10}

m

a_{01}

a_{11}

.....

a_{m-11}

.....

a_{1n}

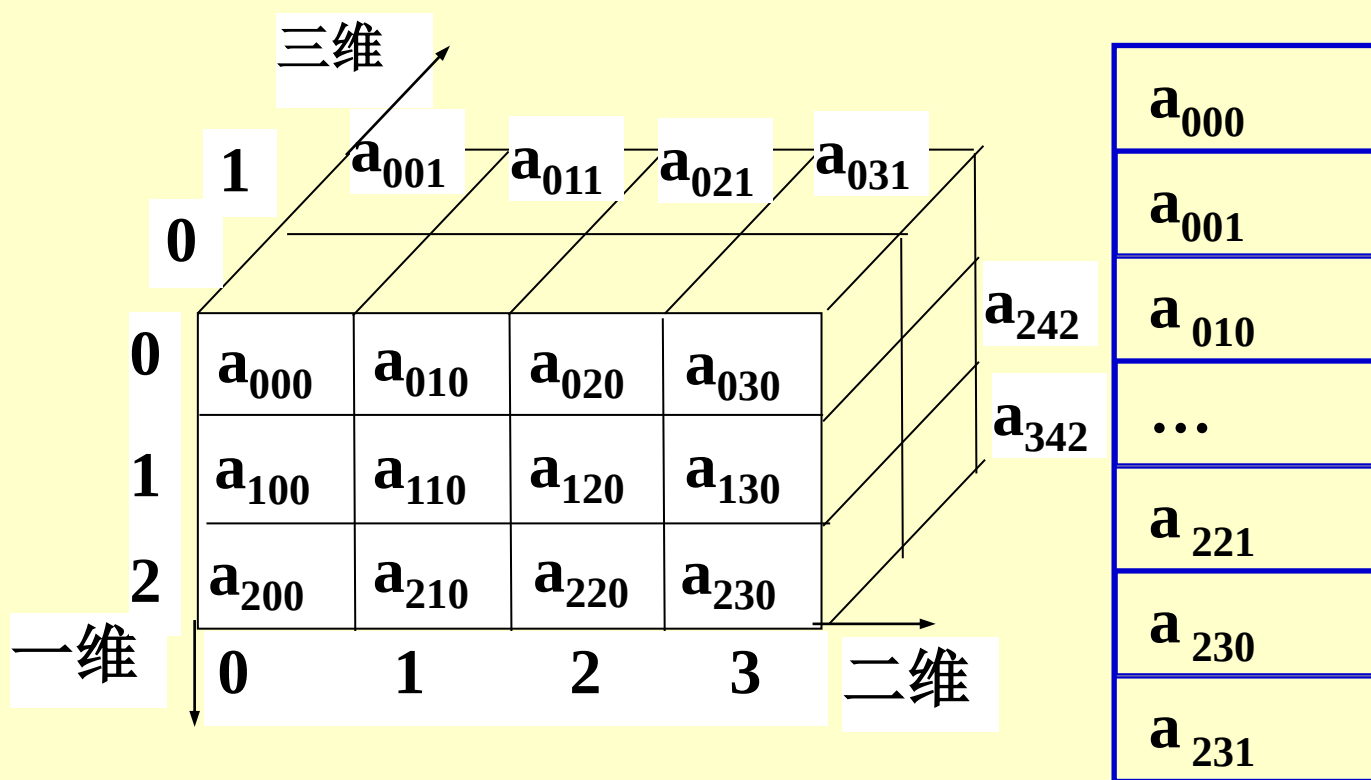
a_{2n}

.....

a_{mn}

规则可以推广到多维数组：**行优先**的优先顺序可规定为前排最右的下标，从右到左，最后排最左下标。

$$\text{LOC}(a_{ijk}) = \text{LOC}(a_{000}) + (i * n * p + j * p + k) * d$$



3*4*2的三维数组的行优先的逻辑结构

返回

- 矩阵是数值程序设计中经常用到的数学模型，它是由 m 行和 n 列的数值构成（ $m=n$ 时为方阵）。
- 通常用二维数组表示矩阵，它使矩阵中的每个元素都可在二维数组中找到相对应的存储位置。
- 在数值分析中，矩阵中有很多值相同的元或零值元。为了节省存储空间，需要对它们进行“压缩存储”，即不存或少存这些值相同的元或零值元。

一、特殊矩阵

特殊矩阵是指非零元素或零元素的分布有一定规律的矩阵。

1、对称矩阵：

定义：在一个n阶方阵A中，若元素满足下述性质：

$a_{ij}=a_{ji}$ $0 \leq i, j \leq n-1$ 则称A为**对称矩阵**。

$$\begin{bmatrix} a_{00} & a_{01} & \dots & \dots & \dots & a_{0n-1} \\ a_{10} & a_{11} & \dots & \dots & \dots & a_{1n-1} \\ \dots & \dots & \dots & \dots & \dots & \dots \\ a_{n-10} & a_{n-11} & \dots & \dots & \dots & a_{n-1n-1} \end{bmatrix}$$

特点：对称矩阵中的元素关于**主对角线对称**，故只要存储矩阵中**上三角或下三角**中的元素，让每两个对称的元素共享一个存储空间，这样，能**节约近一半**的存储空间。

按“行优先顺序”存储主对角线（包括对角线）以下的元素，其存储形式：

对应关系： a_{21} 与 a_{12}

1 5 1 3 7

a_{00}

5 0 8 0 0

a_{10}

a_{11}

$k = \begin{cases} i(i+1)/2 + j & i \geq j \text{ 下三角} \\ j(j+1)/2 + i & i < j \text{ 上三角} \end{cases}$

1 8 9 2 6

a_{20}

a_{21}

a_{23}

3 0 2 5 1

.....

7 0 6 1 3

$a_{n-1\ 0}$

$a_{n-1\ 1}$

$a_{n-1\ 2}$

$\dots a_{n-1\ n-1}$

上三角第 i 行恰有 $i+1$ 个元素，第 i 行以前元素总数为： $i(i+1)/2$

按行序为主序

a_{00}	a_{10}	a_{11}	a_{20}	$\dots$	$a_{n-1\ 0}$	$\dots$	$a_{n-1,n-1}$
----------	----------	----------	----------	---------	--------------	---------	---------------

$k=0$

1

2

3

$n(n-1)/2$

$n(n+1)/2-1$

a_{ij} 的地址计算:

d 为单位空间大小

$$\text{LOC}(a_{ij}) = \text{LOC}(\text{sa}[k])$$

$$= \text{LOC}(\text{sa}[0]) + k * d = \text{LOC}(\text{sa}[0]) + [i * (i+1)/2 + j] * d$$

- 对于任意给定一组下标 (i, j) , 均可在 $\text{sa}[k]$ 中找到矩阵元素 a_{ij} ;
- 对所有的 $k=0, 1, 2, \dots, n(n-1)/2-1$, 都能确定 $\text{sa}[k]$ 中的元素在矩阵中的位置 (i, j) ;
- $\text{sa}[n(n+1)/2]$ 为 n 阶对称矩阵 A 的压缩存储;

a_{00}	a_{10}	a_{11}	a_{20}	...	$a_{n-1,0}$	...	$a_{n-1,n-1}$
----------	----------	----------	----------	-----	-------------	-----	---------------

$k=0$

1

2

3

$n(n-1)/2$

$n(n+1)/2-1$

- a_{21} 和 a_{12} 均存储在 $\text{sa}[4]$ 中, 因为
 $k = i * (i+1)/2 + j = 2 * (2+1)/2 + 1 = 4$

2、三角矩阵

- 三角矩阵：**上三角**和**下三角**。三角矩阵的下三角（不包括主对角线）中的元素均为常数。三角矩阵相反，在大多数情况下，三角矩阵常数为零。

a_{00} a_{01} $\dots$ $a_{0\ n-1}$

c a_{11} $\dots$ $a_{1\ n-1}$

.....

c c $\dots$ $a_{n-1\ n-1}$

(a)上三角矩阵

a_{00} c $\dots$ c

a_{10} a_{11} $\dots$ c

.....

$a_{n-1\ 0}$ $a_{n-1\ 1}$ $\dots$ $a_{n-1\ n-1}$

(b)下三角矩阵

- 三角矩阵中的重复元素 c 可共享一个存储空间，其余的元素正好有 $n(n+1)/2$ 个，三角矩阵可压缩存储到向量 $sa[0..n(n+1)/2]$ 中，其中 c 存放在向量的最后一个分量中。

3、对角矩阵

若n阶方阵中的非零值元都集中在以主对角线为中心的(由k条对角线组成的)带状区域中，则称为k对角矩阵。

$$\begin{bmatrix}
 \mathbf{a}_{00} & \mathbf{a}_{01} & 0 & \dots\dots\dots & 0 \\
 \mathbf{a}_{10} & \mathbf{a}_{11} & \mathbf{a}_{12} & 0 & \dots\dots\dots & 0 \\
 0 & \mathbf{a}_{21} & \mathbf{a}_{22} & \mathbf{a}_{23} & 0 & \dots\dots\dots & 0 \\
 \dots\dots\dots & \dots\dots\dots & \dots\dots\dots & \dots\dots\dots & \dots\dots\dots & \dots\dots\dots & \dots\dots\dots \\
 0 & 0 & \dots & \mathbf{a}_{n-2,n-3} & \mathbf{a}_{n-2,n-2} & \mathbf{a}_{n-2,n-1} \\
 0 & 0 & \dots & \dots & \mathbf{a}_{n-1,n-2} & \mathbf{a}_{n-1,n-1}
 \end{bmatrix}$$

特点:

三对角矩阵的非零元素仅出现在:

- 主对角 $a_{ii}(0 \leq i \leq n-1)$,
- 紧邻主对角线上面的对角线上 $a_{i+1,i}, 0 \leq i \leq n-2$
- 紧邻主对角线下方的对角线上 $(a_{i-1,i}, 0 \leq i \leq n-1)$

显然, 当 $|i-j| > 1$ 时, 元素 $a_{ij} = 0$ 。

存储:

在三对角矩阵, 可按行优序为主序来存储。除第0行和第 $n-1$ 行是2个元素外, 每行的非零元素都要是3个, 需存储的元素个数为 $3n-2$ 。

a_{00}	a_{01}	a_{10}	a_{11}	a_{12}	a_{21}	...	...	$a_{n-1,n-2}$	$a_{n-1,n-1}$
----------	----------	----------	----------	----------	----------	-----	-----	---------------	---------------

$k = 0 \quad 1 \quad 2 \quad 3 \quad 4 \quad \dots \quad \dots \quad 3n-4 \quad 3n-3$

数组sa中的元素sa[k]与三对角带状矩阵中的元素 a_{ij} 存在一一对应关系：

- 在 a_{ij} 之前有i行,共有 $3*i-1$ 个非零元素
- 在第i行当前元素前, 有 $j-i+1$ 个非零元素

非零元素 a_{ij} 的地址为：

$$\begin{aligned}\text{LOC}(i,j) &= \text{LOC}(0,0) + [3*i-1+(j-i+1)]*d \\ &= \text{LOC}(0,0) + (2i+j)*d\end{aligned}$$

上例 a_{34} 对应着sa[10]。

$$k = 2*i + j = 2*3 + 4 = 10$$

a_{21} 对应着sa[5]

$$k = 2*2 + 1 = 5$$

sa[0.. $3*n-1$]是n阶三对角带状矩阵A的压缩存储表示。

二、稀疏矩阵

- **稀疏矩阵**：设矩阵A中有s个非零元素，s远远小于矩阵元素的总数（即 $s \leq m \times n$ ）。
- 设在矩阵A中，有s个非零元素。令 $e = s / (m * n)$ ，称e为矩阵的**稀疏因子**。通常 $e \leq 0.05$ 时称之为稀疏矩阵。
- 在存储稀疏矩阵时，使用压缩存储的方法。一个三元组 (i, j, a_{ij}) 唯一确定了矩阵A的一个非零元。

稀疏矩阵可由表示非零元的三元组及其行列数唯一确定。

i, j从1开始

三元组表

((1,2,12)(1,3,9),(3,1,-3),(3,6,14),(4,3,24), (5,2,18),
(6,1,15),(6,4,-7))

加上(6,7)这一对行、列值便可作为下列矩阵M的另一种描述。
由三元组表的不同表示方法可引出稀疏矩阵不同的压缩存储方法。

$$M = \begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{pmatrix}$$

(行, 列, 元素)

$$T = \begin{pmatrix} 0 & 0 & -3 & 0 & 0 & 15 \\ 12 & 0 & 0 & 0 & 18 & 0 \\ 9 & 0 & 0 & 24 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -7 \\ 0 & 0 & 14 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

(列, 行, 元素)

稀疏矩阵的压缩存储方法

1、三元组顺序表

```
#define number 30
typedef struct{
    int i, j;
    ElemType e;
}triple;
Typedef struct{
    triple data[number];
    int m, n, num;//行数，列数，非零元素个数
}TSMatrix;
```

$$M = \begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{bmatrix}_{6 \times 7}$$

m, n, num分别存放
矩阵行列维数和非
零元个数, 或用第一
行存储这三个值。

三元组表
所需存储
单元个数
为3*num
其中num
为非零元
个数

行列下标

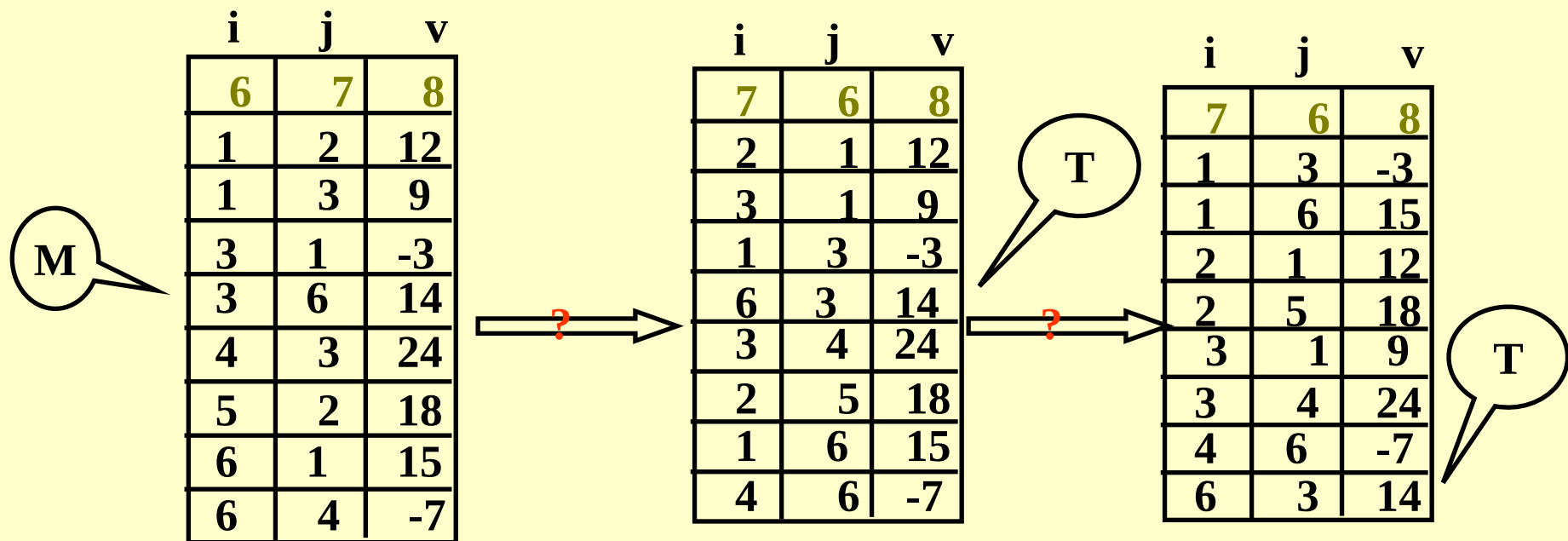
非零元值

	i	j	e
0	6	7	8
1	1	3	9
2	3	1	-3
3	3	6	14
4	4	3	24
5	5	2	18
6	6	1	15
7	6	4	-7

data

求转置矩阵操作 力扣867. 转置矩阵(非稀疏)

- 已知一个稀疏矩阵的三元组表，求该矩阵转置矩阵的三元组表
- 如果只简单地交换M.data中i和j的内容，那么得到的T.data将是一个按列优先顺序存储的稀疏矩阵T，要得到按行优先顺序存储的T.data，就必须重新排列三元组的顺序。



三元组顺序表虽然节省了存储空间，但时间复杂度比一般矩阵转置的算法还要复杂，同时还有可能增加算法的难度，仅适用于 $num \ll m*n$ 的情况。

二维数组存储时矩阵的转置算法:

```
for(col=0;col<n;col++)
```

```
    for(row=0;row<m;row++)
```

```
        n[col][row]=m[row][col];
```

$T(n)=O(m \times n)$

方法一：按M的列序转置，为了找到M中每一列所有非零元素，需对其三元组表从第一行起扫描一遍。由于M中以行序为主序，所以由此得到的恰是T中应有的顺序。

	i	j	e	
	6	7	8	
p → 0	1	2	12	← p
p → 1	1	3	9	← p
p → 2	3	1	-3	← p
p → 3	3	6	14	← p
p → 4	4	3	24	← p
p → 5	5	2	18	← p
p → 6	6	1	15	← p
p → 7	6	4	-7	← p

M

col=1

col=2

M->data[p].j

	i	j	e	
	7	6	8	
q → 0	1	3	-3	
q → 1	1	6	15	
q → 2	2	1	12	
q → 3	2	5	18	
q → 4	3	1	9	
5	3	4	24	
6	4	6	-7	
7	6	3	14	

T

T->data[q].i

方法一：算法

```
Status transposeMatrix(TSMatrix *M, TSMatrix *T){
```

```
    int col, p, q;
```

```
    T->m = M->n;   T->n = M->m;   T->num = M->num;
```

```
    if (T->num) {   q = 0;
```

```
        for (col = 1; col <= _____; col++) {
```

```
            for (p = 0; p < M->num; p++) {
```

```
                if ( _____ == col) {
```

```
                    _____;
```

```
                    _____;
```

```
                    _____;
```

```
                q++; }
```

```
            } } }
```

```
    return OK;
```

```
}
```

算法分析： $T(N) = O(M \text{的列数} n \times \text{非零元个数 num})$

若 num 与 $m \times n$ 同数量级，则 $T(n) = O(m * n^2)$

方法二：快速转置 空间+预计算

按m中三元组次序转置，转置结果放入T中恰当位置. 要预先确定M中每一列第一个非零元在T中位置，为确定这些位置，转置前应先求得M的每一列中非零元个数。

实现：设两个数组：

array[col]：表示矩阵M中第col列中非零元个数

cpot[col]：指示M中第col列第一个非零元在T中位置

显然有：

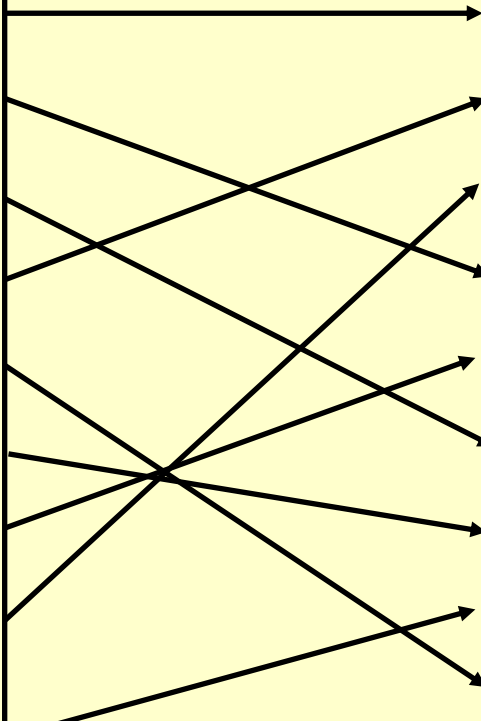
$$\begin{cases} \text{cpot}[1]=1; \\ \text{cpot}[\text{col}]=\text{cpot}[\text{col}-1]+\text{array}[\text{col}-1]; \end{cases} \quad (2 \leq \text{col} \leq \text{M.n})$$

$$M = \begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{bmatrix}_{6 \times 7}$$

col	1	2	3	4	5	6	7
array[col]	2	2	2	1	0	1	0
cpot[col]	1	3	5	7	8	8	9

M

	i	j	e
0	6	7	8
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	6	14
5	4	3	24
6	5	2	18
7	6	1	15
7	6	4	-7



	i	j	e
0	7	6	8
1	1	3	-3
2	1	6	15
3	2	1	12
4	2	5	18
5	3	1	9
6	3	4	24
7	4	6	-7
7	6	3	14

T

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
cpot[col]	1	3	5	7	8	8	9

2

4

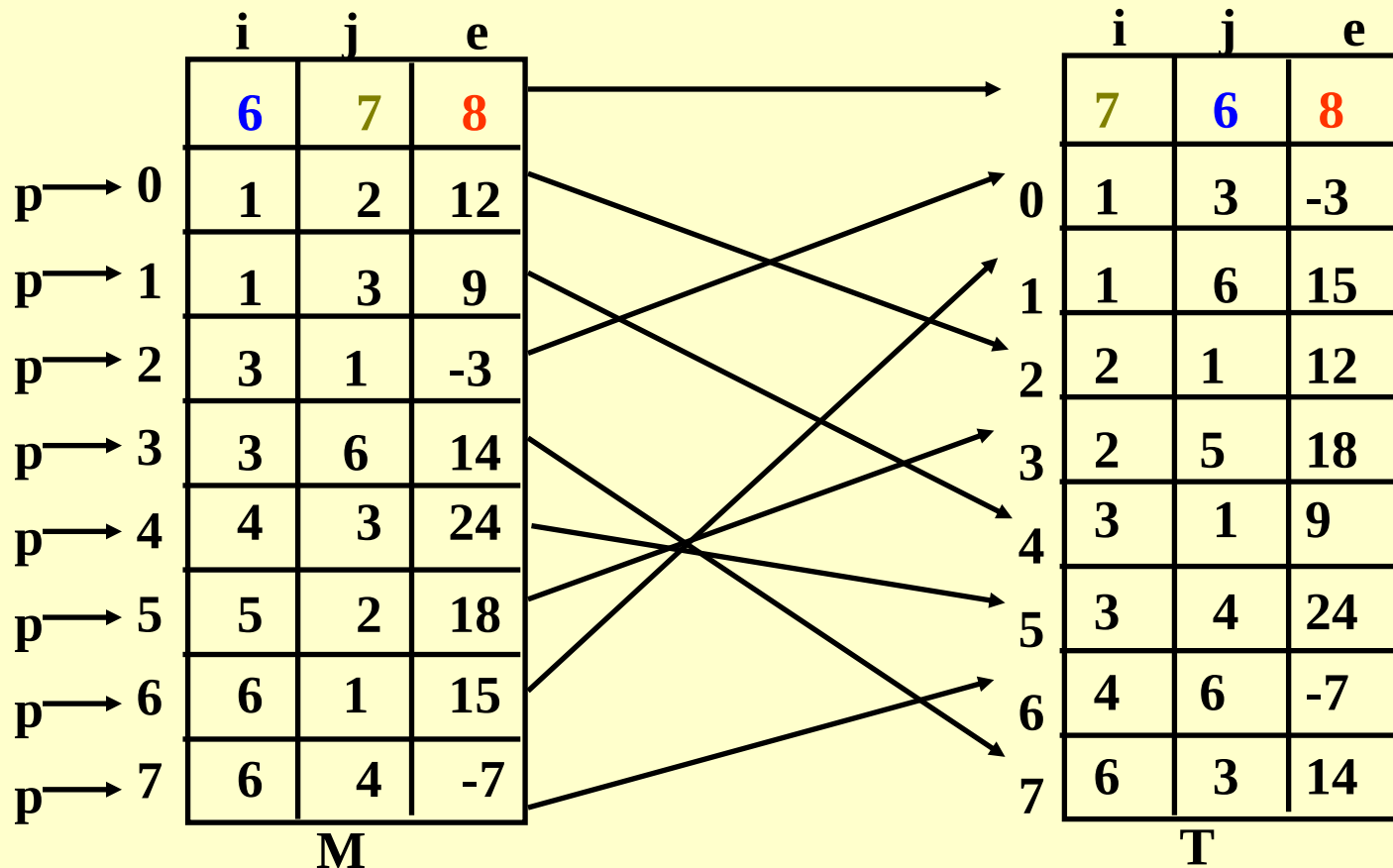
6

9

3

5

7



1 3一定在4 3前面，因此转置后3 1也会在3 4前面

返回

```

Status fastTransposeMatrix(TSMatrix * M, TSMatrix * T){
    int col, t, j, p, q; int *cpot,*array;
    cpot = (int *)malloc(sizeof(int)*(M->n+1));
    array = (int *)malloc(sizeof(int)*(M->n+1));
    T->m = M->n;  T->n = M->m;  T->num = M->num;
    if (T->num) {
        for (col = 1; col <= M->n; col++) {    array[col] = 0;    }
        for (t = 0; t<M->num; t++) {          //统计每列非零元素个数
            j = _____;  array[j]++;
        }
        cpot[1] = 1;
        for (col = 2; col <= M->n; col++) { //M中第col列第一个非零元在T中位置
            cpot[col] = _____;
        }
        for (p = 0; p<M->num; p++) {
            col = M->data[p].j;
            q = cpot[col]; //q-1为T->data数组中第col列的可插入位置
            _____; //M向T中元素赋值
            _____;
            _____;
            cpot[col]++; //位置向下一个
        }
    }
    return OK;
}

```

算法分析: $T(n) = O$
 (M的列数 n +非零元个数
 num)
 若 num 与 $m \times n$ 同数量级,
 则 $T(n) = O(m \times n)$

Experiment 5-1 矩阵转置算法实现

问题:

1. 实现普通转置算法;
2. 实现快速转置算法;

5.4 广义表

- 广义表 (Lists, 又称列表) 是线性表的推广
- 线性表定义为 $n \geq 0$ 个元素 $a_1, a_2, a_3, \dots, a_n$ 的有限序列。线性表的元素仅限于原子项, 原子是作为结构上不可分割的成分, 它可以是一个数或一个结构, 若放松对表元素的这种限制, 容许它们具有其自身结构, 产生了广义表的概念。
- 广义表是 $n (n \geq 0)$ 个元素 $a_1, a_2, a_3, \dots, a_n$ 的有限序列, 其中:
 a_i 或者是原子项, 或者是一个广义表。
- 记作 $LS = (a_1, a_2, a_3, \dots, a_n)$ 。LS 是广义表的名字, n 为它的长度。若 a_i 是广义表, 则称它为 LS 的子表。

表示方法：通常用圆括号将广义表括起来，用逗号分隔其中的元素。为了区别原子和广义表，书写时用大写字母表示广义表，用小写字母表示原子。

若广义表LS ($n \geq 1$)非空，则 a_1 是LS的表头，其余元素组成的表($a_2, \dots, a_n$)称为LS的表尾。

广义表是递归定义的，在定义广义表时又用到了广义表的概念。如：

- (1) $A = ()$ —— A 是一个空表，其长度为零。
- (2) $B = (e)$ —— 表 B 只有一个原子 e ， B 的长度为1。
- (3) $C = (a, (b, c, d))$ —— 表 C 的长度为2，两个元素分别为原子 a 和子表 (b, c, d) 。
- (4) $D = (A, B, C)$ —— 表 D 的长度为3，三个元素都是广义表。将子表的值代入后，则有
 $D = ((), (e), (a, (b, c, d)))$ 。
- (5) $E = (a, E)$ —— 是一个递归的表，它的长度为2， E 相当于一个无限的广义表 $E = (a, (a, (a, (a, \dots))))$ 。

广义表的三个重要结论：

- (1) 广义表的元素可以是子表，而子表的元素还可以是子表。
广义表是一个多层次的结构，可以用图形象地表示。
- (2) 广义表可为其它表所共享。
- (3) 广义表的递归性。

广义表的运算：gethead、gettail

例如： $\text{gethead}(B)=e$ $\text{gettail}(B)=()$

$\text{gethead}(D)=A$ $\text{gettail}(D)=(B,C)$

由于 (B, C) 为非空广义表，则可继续分解得到：

$\text{gethead}(B,C)=B$ $\text{gettail}(B,C)=(C)$

注意广义表 $()$ 和 $(())$ 不同。

前者是长度为0的空表，对其不能做求表头的和表尾的运算；而后者是长度为1的非空表（只不过该表中唯一的一个元素是空表）。对其可进行分解，得到表头和表尾均为空表 $()$ 。

练习与作业

本课练习：习题五 10

本课作业：习题五 13

- 1 广义表 $A=((a,b,c),(d,e,f))$,
 $\text{head}(\text{tail}(\text{head}(\text{tail}(A))))$ 是什么?
- 2 广义表 $((((a,b,()),c),d),e,((f),g))$ 的长度是____, 深度是____。
- 3 $C=(a,(b,c),d)$ 这个表尾是多少?

1、设数组a[7][6]的基地址为1024,每个元素占2个存储单元,若以行序为主顺序存储,则元素a[2][4]的存储地址是__。(中)

A、1058 B、1056 C、1098 D、答案A,B,C都不对

2、二维数组A中, 每个元素的长度为3个字节, 行下标i从0到7, 列下标j从0到9, 从首地址SA开始连续存放在存储器内, 存放该数组至少需要的字节数是____。(中)

A、80 B、100 C、240 D、270

3、二维数组A中, 每个元素A的长度为3个字节, 行下标i从0到7, 列下标j从0到9, 从首地址SA开始连续存放在存储器内, 该数组按行存放时, 数组元素A[7][4]的起始地址为____。(中)

A、SA+141 B、SA+144 C、SA+222 D、SA+225

4、对n阶对称矩阵压缩存储时, 需要表长为()的顺序表。

A、 $n/2$ B、 $n*n/2$ C、 $n(n+1)/2$ D、 $n(n-1)/2$

5、对n阶对称矩阵A,将其下三角部分按行存放在一维数组B中, 而A[0][0]存放于B[0]中,则第i行的对角元素A[i][i]存放于B中的()处。

A、 $(i+3)i/2$ B、 $(i+1)i/2$ C、 $(2n-i+1)i/2$ D、 $(2n-i-1)i/2$

6、【2016统考真题】有一个100阶的三对角矩阵M, 其元素 $m_{i,j}$ ($1 \leq i, j \leq 100$) 按行优先依次压缩存入下标从0开始的一维数组N中。元素 $m_{30,30}$ 在N中的下标是()。

A、86 B、87 C、88 D、89

7、【2017统考真题】适用于压缩存储稀疏矩阵的两种存储结构是（ ）。

A、三元组表和十字链表 B、三元组表和邻接矩阵

C、十字链表和二叉链表 D、邻接矩阵和十字链表

8、【2018统考真题】设有一个 12×12 的对称矩阵M，将其上三角部分的元素 $m_{i,j} (1 \leq i \leq j \leq 12)$ 按行优先存入C语言的一维数组N中，元素 $m_{6,6}$ 在N中的下标是（ ）。

A、50 B、51 C、55 D、66

9、已知广义表 $L = ((x, y, z), a, (u, t, w))$ ，从L表中取出原子项t的运算是（ ）。

A. $\text{head}(\text{tail}(\text{tail}(L)))$ B. $\text{tail}(\text{head}(\text{head}(\text{tail}(L))))$

C. $\text{head}(\text{tail}(\text{head}(\text{tail}(L))))$ D. $\text{head}(\text{tail}(\text{head}(\text{tail}(\text{tail}(L))))$

10、广义表 $A = (a, b, (c, d), (e, (f, g)))$ ，则下面式子的值为（ ）。

$\text{Head}(\text{Tail}(\text{Head}(\text{Tail}(\text{Tail}(A)))))$